DATA STRUCTURES AND ALGORITHMS

ASSIGNMENT 1: TIME COMPLEXITY AND BIG-O NOTATION

Answer each question and justification. Where a complexity is identified, explain the reasoning used to determine it.

Part A - Foundations of Time Complexity

1. A school has 20 student records today and may have 20,000 records in future. Why is it not enough to say that an algorithm is "fast"? What should be considered as the amount of data grows?

2. Explain the difference between **actual running time** measured in seconds and **time complexity**. Could the same algorithm run in different numbers of seconds on two computers but still have the same time complexity? Explain.

3. In algorithm analysis, what does **n** represent? Give three different problems in which n could represent different things.

Part B - Complexity in Practical Situations

4. A lecturer has a class list arranged by student number. Compare these two tasks: (a) opening the record at a known position, and (b) reading every student's record to calculate the class average. How does the work in each task change when the class grows from 30 to 300 students? Identify the likely complexity of each.

5. A person is looking for a name in an unsorted list by checking names one after another. Another person searches a sorted list by repeatedly checking the middle and eliminating half of the remaining list. Which approach scales better? Explain **why**, not only the Big-O symbols.

6. Suppose every student in a group must compare an answer with every other student. What happens to the amount of comparison work as the group becomes larger? Why can this type of work grow much faster than simply checking each student once?

Part C - Code Analysis

7. Study the following code.

```
int value = numbers[6];
```

If the array grows from 10 elements to 1,000,000 elements, does the work required by this statement grow with n? State and justify its complexity.

8. Study the following code.

```
for (int i = 0; i < n; i++) {
    cout << numbers[i] << endl;
}
```

For n = 10, about how many array elements are visited? For n = 1,000? What pattern do you observe? State and justify the complexity.

9. Study the following two loops.

```
for (int i = 0; i < n; i++) {
    cout << i;
}
```

```
for (int j = 0; j < n; j++) {
    cout << j;
}
```

A student says, "There are two loops, so the complexity must be $O(n^2)$." Is the student correct? Explain by estimating the total work.

10. Now study the nested loops.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cout << i << " " << j;
    }
}
```

For n = 5, approximately how many times does the inner statement execute? What about n = 100? Explain why this differs from Question 9 and state the complexity.

Part D - Complexity of Array Operations

11. An array has capacity 1,000 and currently contains 700 logical elements. Compare these operations: accessing element 450; inserting a new value at the end when free capacity exists; inserting a new value at index 0. Which operation may require the most work, and why?

12. Deleting the final logical element of an array can be very different from deleting the first element. Explain the difference in the work required. What are the likely complexities?

13. Why is it incorrect to say, "An array is O(1)" or "An array is O(n)" without naming an operation? Use at least three array operations to support the answer.

Part E - Search Algorithms and Case Analysis

14. Consider linear search on an array of n elements. Describe a situation representing the **best case** and another representing the **worst case**. How many comparisons might each require?

15. A sorted array contains 1,024 values. Linear search may require up to about 1,024 checks. Binary search can reduce the remaining search area by half each time. Estimate how many halvings binary search needs to reduce 1,024 possibilities to about one. What does this reveal about O(n) versus O(log n)?

16. Why can binary search not simply be applied in the normal way to an unsorted array? Explain what information sorting gives after a comparison with the middle value.

Part F - Big-O Analysis

17. An algorithm performs approximately **3n + 10** operations. Another performs approximately **50n + 2**. What Big-O class describes each? Why are the constants normally removed in Big-O notation?

18. Consider the expression $\mathbf{n^2 + 5n + 20}$. Which term controls the growth when n becomes very large? What Big-O complexity results? Explain rather than only simplifying the expression.

19. Arrange the following from the slowest-growing to the fastest-growing as n becomes large: $\mathbf{O(n^2)}$, **O(1), O(n), O(log n),** $\mathbf{O(2^n)}$, **O(n log n)**. Explain what happens to the gap between them as n increases.

20. Is an O(n) algorithm always faster than an $O(n^2)$ algorithm for every possible small input and every implementation? Explain what Big-O actually allows us to conclude.

Part G - Applied Problem Analysis

21. Choose one real problem: finding a student record, searching for a book, processing examination marks, finding the highest mark, arranging records, or another computing problem familiar to the group.

Describe the input and define what **n** means. Propose an algorithm. Explain the main operation being repeated. Estimate its time complexity. Then discuss whether a different organisation of the data or a different algorithm could improve the complexity.

Reference Table

Complexity	Name	Growth idea
O(1)	Constant	Work does not grow with n
O(log n)	Logarithmic	Problem repeatedly reduces, often by half
O(n)	Linear	Work grows roughly with n
O(n log n)	Linearithmic	Linear work across logarithmic stages
$O(n^2)$	Quadratic	Work can grow roughly as n x n

Complexity	Name	Growth idea
O(2^n)	Exponential	Work can roughly double when n increases by 1